



Desarrollo Web Profesional

con HTML, CSS, JavaScript y Git

Sesión 04

Tarjetas con Grid y la primera línea de JavaScript

Ing. Luis Carlos Contreras Zelada

lc@zolar.dev

github.com/jinchuika/desarrollo-web

Colegio de Ingenieros de Guatemala

~ Revisión en vivo, por sorteo



Dos participantes por sesión, tres minutos cada uno. Se revisa el código, no a la persona.

- 🌐 **Se abre la dirección pública.** La que empieza con el usuario de GitHub, no la ruta de la computadora.
- ☰ **Se aplica la lista de la sesión 3.** Punto por punto, en voz alta, sobre la hoja de estilos.
- ⌕ **Se pregunta por una línea.** Elegida del archivo. Qué hace, por qué está ahí, qué pasa si se borra.
- ↔ **Se pide un cambio en vivo.** Cambiar el color de acento del sitio completo. Con variables es una línea.
- 🔗 **Se lee el historial.** Commits repartidos en el tiempo, con mensajes que dicen qué cambió.

Quien no registró su dirección no entra al sorteo, y sin dirección registrada no hay nada que calificar.

La tipografía que no llegó

Se definió en la identidad, se escribió en la hoja y el navegador siguió usando otra letra.

Por qué pasa

Una variable solo guarda un nombre. Si el archivo de la tipografía nunca se descarga, el navegador usa la primera de la lista que sí tenga instalada.

Cómo se corrige

Se agrega el enlace de Google Fonts en el head de las tres páginas, antes de la hoja de estilos. El nombre debe escribirse igual que en Google Fonts.

```
<link rel="stylesheet" href="https://fonts.googleapis.com/css2?family=Montserrat:wght@400;700&display=swap">
```

```
<link rel="stylesheet" href="css/estilos.css">  
/* en la hoja, el nombre exacto y una alternativa */  
--fuente-titulos: "Montserrat", Arial, sans-serif;
```

Se verifica en la pestaña de red: si la tipografía no aparece descargada, no se está usando.

Doble clic o servidor local



Las dos formas abren la página. Solo una se parece a lo que pasa cuando está publicada.

Doble clic sobre el archivo

La dirección empieza con file. El navegador trata cada archivo como suelto y limita lo que puede cargar. Sirve para ver texto y estilos, poco más.

Live Server

La dirección empieza con http y lleva un puerto. Es un servidor en su propia computadora: se comporta como el sitio publicado y recarga solo al guardar.

```
file:///C:/Users/nombre/Documents/GitHub/mi-sitio/perfil.html
http://127.0.0.1:5500/perfil.html
# clic derecho sobre el archivo, Open with Live Server
```

De hoy en adelante se trabaja con Live Server. JavaScript lo necesita.



Flexbox o Grid

Una fila o dos direcciones a la vez. La misma regla acomoda cualquier ancho de pantalla.

Flexbox: una dirección



```
display: flex;
```

Grid: dos direcciones



```
display: grid;
```

```
.tarjetas {  
  display: grid;  
  grid-template-columns:  
    repeat(auto-fit, minmax(240px, 1fr));  
  gap: 1.5rem;  
}
```

Tantas columnas como quepan con 240px de mínimo. El número de columnas lo decide el ancho de pantalla, no el código.

Su menú ya usa Flexbox. Hoy se agrega Grid para las tarjetas.

Tarjetas con Grid

La sección que se agrega hoy a perfil.html. Así debe quedar lo que genere la IA.

```
<section>
  <h2>Proyectos</h2>
  <div class="tarjetas">
    <article class="tarjeta">
      <h3>Nombre del proyecto</h3>
      <p>Que se hizo y para quien.</p>
    </article>
    <!-- una article por tarjeta -->
  </div>
</section>
/* === TARJETAS === */
.tarjetas {
  display: grid;
  grid-template-columns:
    repeat(auto-fit, minmax(240px, 1fr));
  gap: 1.5rem;
}
```

Una article por tarjeta. Cada tarjeta se entiende sola, fuera de la página.

div para agrupar. Solo acomoda las tarjetas. Es el uso correcto del div.

La cuadrícula va en el contenedor. display: grid se escribe en .tarjetas, no en .tarjeta.

gap, no margin. El espacio entre tarjetas lo pone la cuadrícula.

La instrucción para la IA

Contexto, tarea y restricciones. Se adjuntan perfil.html, css/estilos.css e identidad.txt.

Contexto: estoy construyendo mi sitio personal con HTML y CSS, sin librerías.
Adjunto perfil.html, css/estilos.css e identidad.txt. Los colores ya están como variables en :root.

Tarea: agrega a perfil.html una sección "Proyectos" con 3 tarjetas.
Cada tarjeta lleva un título h3 y un párrafo. Dame también el CSS.

Restricciones:

- una etiqueta article por tarjeta, dentro de un div con class="tarjetas"
- CSS Grid con repeat(auto-fit, minmax(240px, 1fr)) en el contenedor
- usa solo las variables de :root; no agregues colores nuevos
- sin !important, sin estilos en línea y sin etiquetas style
- no cambies las reglas que ya existen; entrega solo las nuevas
- responde solo con los dos fragmentos de código

El contenido de las tarjetas lo escribe usted, con proyectos reales. La IA arma la estructura.

El diff no reemplaza la lectura

El control de versiones dice qué líneas cambiaron. Decir si están bien sigue siendo trabajo suyo.



Colores fuera de :root. Un código de color escrito dentro de una regla nueva rompe la identidad.

Reglas que ya existían, reescritas. Se pidió solo lo nuevo. Si el diff toca el encabezado, hay que devolverlo.

Clases con nombre de apariencia. .tarjeta-azul, .caja-grande. El nombre dice qué es, no cómo se ve.

Medidas fijas y !important. Un ancho en píxeles dentro de la tarjeta anula la cuadrícula.

Etiquetas equivocadas. div donde iba article, h4 donde iba h3, tarjetas fuera del main.

Cada defecto encontrado se anota en la bitácora, con lo que se corrigió a mano.



Qué agrega JavaScript

Tres lenguajes, tres trabajos distintos, sobre el mismo documento.

HTML

Estructura

Qué hay en la página y qué significa cada parte.

CSS

Apariencia

Cómo se ve y cómo se acomoda en cada ancho.

JavaScript

Comportamiento

Qué pasa cuando alguien hace algo, y qué cambia después.

El navegador ya trae el intérprete. Para JavaScript no se instala nada.

Dónde vive el script

Un archivo aparte y una línea de enlace, igual que la hoja de estilos.

```
mi-sitio/  
  index.html  
  perfil.html  
  contacto.html  
  css/estilos.css  
  js/scripts.js  
<!-- en el head de las tres paginas -->  
<script src="js/scripts.js" defer></script>
```

Un solo archivo. js/scripts.js para todo el sitio, como css/estilos.css.

defer. Espera a que el documento esté completo antes de ejecutar. Sin esto, el script busca elementos que todavía no existen.

Ruta relativa, en minúsculas. js/scripts.js, no JS/Scripts.js. El servidor distingue mayúsculas.

Sin código en el HTML. Ninguna etiqueta script con código adentro, ninguna función en un atributo.

La consola

La misma ventana de la pestaña de red, en la primera pestaña. Es donde el navegador avisa.

```
console.log("el script se esta ejecutando");  
# lo que aparece cuando algo falla:  
Uncaught TypeError: anio is null  
    scripts.js:3  
# la linea 3 de scripts.js busco un elemento  
# que no existe en esa pagina
```

F12, o clic derecho e inspeccionar. La pestaña se llama consola.

El error dice archivo y línea. Se lee de izquierda a derecha y se abre esa línea.

console.log es un instrumento. Sirve para confirmar que una línea se ejecutó y con qué valor.

Se borra antes de publicar. Un console.log olvidado es código muerto.

Seleccionar y responder

Dos instrucciones sostienen casi todo lo que se escribe en este curso.



```
// 1. buscar el elemento en el documento
const boton = document.querySelector("#ver-mas");
// 2. decir que hacer cuando alguien lo presiona
boton.addEventListener("click", function () {
  console.log("presionaron el boton");
});
```

querySelector usa selectores de CSS. Los mismos de la sesión pasada: #id, .clase, nav a.

addEventListener escucha. Recibe el nombre del evento y lo que debe pasar cuando ocurre.

const guarda el elemento. Un nombre para no volver a buscarlo en cada línea.

Si el elemento no existe, falla. El id del HTML y el del script tienen que ser idénticos.

El primer comportamiento



El año del pie de página deja de escribirse a mano. Tres líneas que se pueden explicar.

```
<!-- en el footer de las tres paginas -->
<p>&copy; <span id="anio"></span> Luis Carlos Contreras</p>
// js/scripts.js
const anio = document.querySelector("#anio");
anio.textContent = new Date().getFullYear();
```

El span está vacío. El HTML solo reserva el lugar donde va a ir el año.

textContent escribe el texto.
Reemplaza lo que haya dentro del elemento.

new Date() pregunta la fecha. La del dispositivo de quien visita, no la del servidor.

Tres líneas propias valen más que treinta generadas que no se pueden explicar.



Mostrar y ocultar

JavaScript no pinta nada. Agrega o quita una clase, y el CSS decide cómo se ve.

```
<button id="ver-mas" type="button">Ver mas
proyectos</button>
<div id="proyectos-extra" class="oculto"> ... </div>
/* css/estilos.css */
.oculto { display: none; }
// js/scripts.js
const boton = document.querySelector("#ver-mas");
const extra = document.querySelector("#proyectos-extra");
boton.addEventListener("click", function () {
  extra.classList.toggle("oculto");
});
```

La clase empieza puesta. oculto en el HTML esconde el bloque desde el principio.

toggle la quita o la pone. Cada clic invierte el estado: si está, la quita; si no está, la agrega.

El CSS ya tenía la regla. display: none se escribió en la sesión 3. JavaScript solo activa esa regla.

JavaScript decide

Hasta ahora el script siempre hace lo mismo. Con if se le da una condición.

```
<p><span id="saludo"></span>, gracias por visitar</p>
// js/scripts.js
const saludo = document.querySelector("#saludo");
const hora = new Date().getHours();
if (hora < 12) {
  saludo.textContent = "Buenos dias";
} else if (hora < 19) {
  saludo.textContent = "Buenas tardes";
} else {
  saludo.textContent = "Buenas noches";
}
```

getHours() devuelve un número. 0 a 23, la hora del dispositivo de quien visita.

if evalúa una condición. Si es verdadera, ejecuta ese bloque y se salta el resto.

else if encadena otra condición. Se revisa solo si la anterior fue falsa.

else cubre lo que sobra. Ninguna condición sin él; siempre hay que decidir el caso por defecto.

Lista de verificación

Se aplica antes de publicar, y es la lista con la que se revisa en la sesión 5.



Las tarjetas están dentro de un contenedor con display grid, no una por una.

Ninguna tarjeta tiene ancho fijo en píxeles.

Los colores nuevos salen de las variables de :root.

Los nombres de clase dicen qué es el elemento, no cómo se ve.

El script está enlazado con defer en las tres páginas.



La consola no muestra errores en ninguna de las tres.

No quedaron console.log de prueba.

La bitácora tiene una fila por cada generación, con lo corregido a mano.

Se verifica en el sitio publicado y en el teléfono, con datos móviles.



Con estilos, el sitio se ve. Con JavaScript, el sitio responde.

Para la siguiente sesión: anotar en el teléfono lo que se ve mal en las tres páginas. Esa lista es el punto de partida del diseño adaptable y del formulario con validación.



auto-fit y minmax

Una sola regla sirve para la computadora y para el teléfono. No hay una versión para cada pantalla.



Pantalla ancha



Tres columnas

Tableta



Dos columnas

Teléfono



Una columna

minmax(240px, 1fr) fija el mínimo de cada columna y deja que crezca hasta repartir el espacio sobrante.

repeat(auto-fit, ...) crea tantas columnas como quepan con ese mínimo, sin decirle cuántas son.

El número de columnas lo decide el ancho disponible, no el código.

Flexbox o Grid

Los dos acomodan elementos. La diferencia es cuántas direcciones controlan a la vez.

Flexbox: una dirección

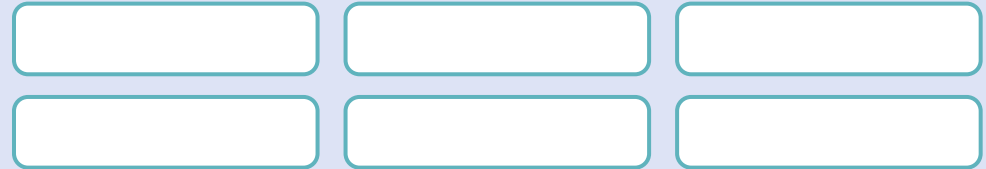
Una fila o una columna. El menú, los botones, el encabezado.



```
display: flex;  
gap: 1rem;
```

Grid: dos direcciones

Filas y columnas a la vez. Las tarjetas, una galería.



```
display: grid;  
gap: 1.5rem;
```



Su menú ya usa Flexbox. Hoy se agrega Grid para las tarjetas.